

Local Life Agent 设计说明

不是返回一个 POI 列表，而是把自然语言生活需求拆成有顺序的任务，逐项调用工具，并生成可确认、可执行、可异常重规划的完整行程。

1. Planning 策略与主流程

API 入口：POST /api/plan 与 POST /api/plan/stream 都进入 PlannerAgent.run；SSE 只展示进度。重规划确认入口 POST /api/replan/confirm 只在用户选择方案后更新计划副本。

自然语言输入	Task Planning	Tool Routing	Candidate Selection	Itinerary Composition	Exception Detection	Consent & Replan
Intent Parsing 时间/同行/child_age/距离/饮食/预算/避免条件	Ordered Tasks + time_window + route_needed	POI / Restaurant / Food Order	相关性验证 → 任务感知排序 → 用户偏好微调	Route / Timeline / Actions / Plan Steps	餐厅无座 / 活动不可用 / 交通或时间冲突	用户确认后再更新计划
策略		当前项目如何实现		解决的问题		
结构化意图解析		LLM JSON 优先，失败回到规则解析；提取场景、时间、同行/人、child_age、距离、饮食、预算、天气和避免条件。		把口语化需求转成可校验约束。		
Ordered Task Planning		LLM 只拆任务和排序，不编造地点；Pydantic 校验后归一 task_id、tool_name、route_needed；规则 planner fallback。		避免多任务被压成单一活动/餐厅。		
Task-level Context		apply_task_context 按任务分句挂载 companions、child_age、constraints；成人酒吧任务不继承儿童约束。		防止全局约束污染无关任务。		
Task-aware Selection		validate_candidate 先过滤弱相关候选；rank_task_candidates 再按相关性、距离、评分、等待、偏好、时段、预算、室内外、儿童适配排序。		不直接采用 AMap 第一条。		
Structured Itinerary		compose_itinerary 只用 ToolExecutionResult 组装 Route、Timeline、Actions；外卖进入 Timeline/Actions，但 route_needed=false，不进线下 Route。		保证摘要、消息和动作时间一致。		

2. 工具调用链路与 Provider 边界

Agent / Planning PlannerAgent、llm_intent_parser、llm_task_planner、ToolRouter、compose_itinerary、final_composer 编排主流程，维护任务顺序，生成结构化计划并校验最终文案。	Tool Layer poi_tool、restaurant_tool、food_order_tool、route_tool、reservation_tool、message_tool 搜索候选、模拟动作、估算路线，统一输出 ToolExecutionResult / ActionResult。
Provider Layer DeepSeekProvider、AmapProvider、local_commerce_provider、本	关键代码依据 app.py 是 API 入口；PlannerAgent.run 是唯一主编排；ToolRouter

地 JSON 构成外部 API 与 fallback 边界；AMap 只提供 geocode/place/route，交易字段由 Mock Commerce 补齐。	完成 task-to-tool；result_validator/task_ranker 先相关再排序；compose_itinerary/final_composer 保证 Route、Timeline、Actions、Summary 与 Share Message 一致。
--	--

3. 异常处理闭环

detect_exceptions 只产出 PlanException，不修改当前计划；build_replan_proposals 生成选项；用户在 ExceptionConfirmationCard 中接受或保留后，apply_replan 才更新计划副本。

异常	检测条件	Replan 方案
restaurant_full	Mock 预约失败、详情包含 restaurant_full，或 Demo/Hybrid 餐厅满员场景。	更换低等待餐厅并更新路线、时间线、模拟预约、分享消息；或保留原餐厅。
activity_unavailable	Demo 场景模拟活动售罄、场馆关闭或 unavailable。	更换同类满足原约束的活动；无合格备选时提示放宽距离或类型，不硬选无关地点。
schedule_conflict	交通延误后预计到达晚于预约时间，影响后续安排。	调整模拟预约时间、切换低等待餐厅，或保持原计划并显示风险。

统一流程：Detect → Assess Impact → Generate Options → User Consent → Apply Replan。接受替代地点时同步更新 RouteStop、Timeline/Plan Step、Mock 预约、Share Message 和 natural_language；保留原计划只记录风险。

4. Fallback、运行模式与 Mock 边界

模式	查询与规划	执行动作	用途
Demo	强制 Rule/Mock LLM、本地 POI/餐厅/路线数据。	Mock	零 Key、离线评审、稳定异常场景。
Hybrid	配置的真实 LLM Provider（当前 DeepSeekProvider / OpenAI-style）与 AMap；失败自动 fallback。	Mock	比赛视频与主要展示。
Live	打开真实 LLM/AMap 查询 Provider。	未实现真实交易时标记 mock_fallback。	未来扩展真实 Action Provider。

LLM 失败回到规则 Intent/Task Planner；AMap 失败、超时、限流、空结果或无效响应回到本地候选和离线路线估算。当前点餐、预约、消息都是 Mock Action，不代表真实下单、支付、订座或消息发送；未实现真实购票、打车、实时库存或天气动态推送。

5. 输出与技术亮点

Plan Summary + Ordered Tasks + Route + Timeline + Replan Proposal + Mock Action Status + Share Message + Explainable Agent Trace

普通用户优先看到最终方案、时间线、地图、Mock Action 状态和分享消息；Agent Trace、候选过滤原因、Provider source 和 fallback 细节默认折叠，评委可展开查看。

Task-first Planning 先拆解并执行有序任务，而不是直接返回推荐列表。	Task-aware Tool Chain 任务相关性优先，偏好、距离、等待和预算作为二级排序。	Consent-based Recovery 异常不会直接破坏计划，用户确认后才重规划。
---	--	--